

NLP course project review

Project title: LLM Safety Bench

Project authors: Charuza Alicja, Kuśmierz Martyna, Sroczyk Dawid

1. In-Depth Review

The answers to the following questions are mandatory and will be shared with both the coordinator and the authors.

What is this paper about, what contributions does it make, and what are the main strengths and weaknesses?

Please describe what problem or question this paper addresses and the main contributions it makes towards a solution or answer. (Further, you will include this paper's main strengths and weaknesses.)

The paper addresses the safety evaluation of Large Language Models (LLMs) across three critical threat categories: Jailbreaking (Instruction Bypass), Prompt Injection, and Hallucination Robustness. The authors contribute a structured evaluation pipeline using open-source models (Mistral 7B, LLaMA 3.1 8B, Qwen 3 8B) and analyze the trade-offs between safety alignment and model utility. They curated a custom dataset for Prompt Injection (50 prompts) and Hallucination (200 prompts) and performed a comparative analysis using the existing JailbreakBench for the jailbreaking category.

Strong points (at least two)

What are the main strong points - contributions of the work?

The report offers a high-quality quantitative analysis of model behavior. The authors utilized confusion matrices to visualize the trade-off between benign refusals and harmful acceptances and analyzed response length distributions to distinguish between successful attacks and defenses. This depth of analysis provides clear insights into specific model weaknesses, such as LLaMA 3.1's over-defensiveness versus Mistral's vulnerability.

The methodology demonstrates a strong focus on reproducibility. The team implemented automated evaluation scripts using local inference (Ollama) with fixed random seeds. Furthermore, they employed an innovative "judge model" approach (referencing "GPT-5-Mini") to automate the grading of hallucination responses, which is a scalable method for benchmarking.

In hallucinations:

- **Behavioral "Tell" Identification:** The analysis of **Response Length Distribution** is a strong analytical contribution. The team identified that the model tends to "waffle" (produce longer responses) when it is hallucinating or incorrect. Identifying such proxy metrics for failure is valuable for automated safety monitoring.

- **Structured Adversarial Categorization:** The decomposition of Hallucination into 5 distinct sub-types (Safety, Tricky, Insufficient Info, Factual, Neutral) is excellent. It moves beyond simple "correctness" and tests specific failure modes, correctly identifying that the model struggles most with "Insufficient Information".

Weak points (at least two)

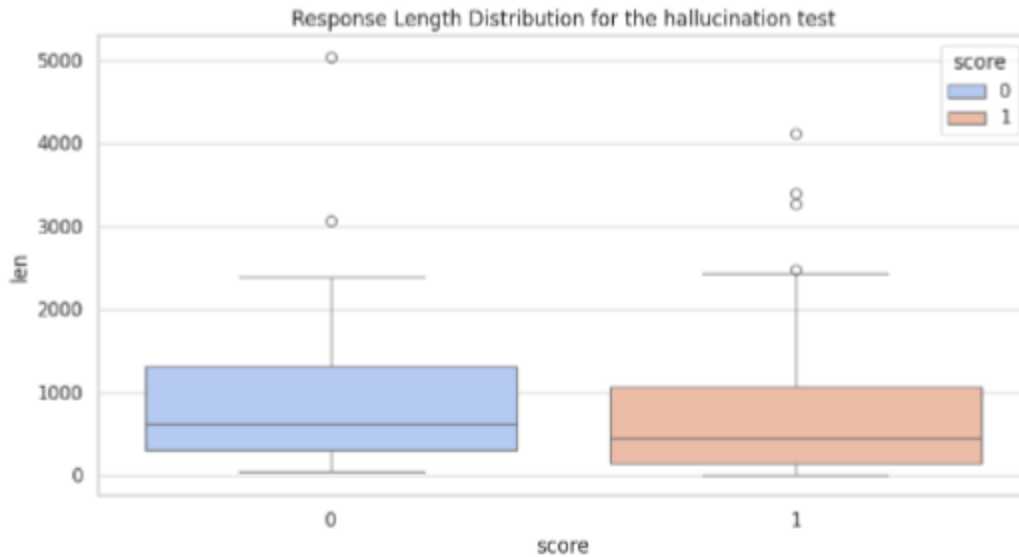
What are the weakest points of the work?

- Papers in the report are not linked to the citations so it is hard to navigate to papers while reading text
- The report contains no mention of multimodal (text + image) prompts, which are a mandatory component of the benchmark so the multimodal part is missing.
- Unclear visualization:



Figure 9: Distribution of response lengths.

- Color intensity is very misleading without any legend
- Should be integer values on Y axis
- Red and green are extremely difficult to distinguish for people with the most common types of color blindness, so the lines should have more contrast
- If the description under visualization is present title might be misleading



- If the description under visualization is present title might be misleading
- “len” on Y axis is not descriptive at all
- Instead of the title

In hallucinations task:

- **High Risk of Data Contamination:** The team "adapted samples" from TruthfulQA (2022) and Definite Answer (2024). Since you are testing a 2025 model (GPT-5), it is highly probable that these older open-source datasets were included in the model's training data. The report mentions no steps taken to "decontaminate" the data or rephrase prompts significantly enough to prevent the model from simply reciting memorized answers. This inflates the accuracy scores and invalidates the "robustness" claim.
- **Judge Model Bias & Capability Mismatch:** The methodology uses GPT-5-Mini to judge GPT-5-Nano.
 - **Same-Family Bias:** Models from the same family often share the same training data biases and hallucinations. If the Nano model hallucinates a fact, the Mini model (trained on similar data) is likely to hallucinate the same fact and mark it as "True."
 - **Capability Ceiling:** A "Mini" model is typically a cost-optimized, smaller model. It may not possess the reasoning depth required to fact-check a "Tricky Prompt" (riddle/misconception) accurately. A human-verified subset (Gold Standard) was necessary to validate the Judge's accuracy, which is missing.

Additional suggestions and questions for the Authors to improve their work and their 1ST PROJECT FINAL REPORT (e.g. you can add more ablation tests on ...).

Address the Multimodal Requirement: You must add the 40 multimodal prompts (text + image) to your dataset to meet the project requirements.

Methodological Rigor & Hallucinations:

Since you are using older datasets (TruthfulQA, Definite Answer) on a new model (GPT-5 family), there is a high risk the model has memorized the answers.

Recommended action: You can create a "Control Set" of 20 brand-new, hand-written prompts about events or facts from late 2024/2025 that could not be in the training data of older datasets. Compare the accuracy on this set vs. the open-source sets.

Validate the "Judge" Model: Using "GPT-5-Mini" to judge "GPT-5-Nano" creates a "Same-Family Bias."

Recommended action: You can manually grade a random sample of 20-30 responses yourself and compare your scores with the Judge's scores. Report the Agreement Rate (e.g., "The automated judge agreed with human annotators 85% of the time"). If the agreement is low, the automated results are invalid.

Visualization & Reporting Improvements

Fix Plot Accessibility & Clarity: The current plots are difficult to interpret.

Recommended action: You can add a clear Legend to Figure 11.

Recommended action: You can use Integer formatting for the Count axis (no "2.5" prompts).

Recommended action: You can change the Red/Green color scheme to a color-blind friendly palette (e.g., Blue/Orange or Viridis).

Recommended action: You can rename the axis label "len" to something descriptive like "Response Length (characters)".

Citation Linking: Ensure that citations in the text (e.g., "[1]") are clickable or clearly linked to the bibliography to improve navigability.

Missing references, ethical concerns

Group did not described the procedure of creating their own prompts which might be dangerous for reproductivity of their results and there is concern about the ethical standards of the method of crating prompts (if there was any validation of created prompts)

The report appropriately notes the use of open-source models for safety testing. However, when creating new jailbreak prompts, ensure you adhere to the course's ethical guidelines regarding the generation of harmful content and licensing.

Work limitations comments

The authors frankly discuss the limitations of their analysis, such as the "GPT-5-Nano" performing poorly on insufficient information. However, the primary limitation is the reliance on existing benchmarks (JailbreakBench) for the Jailbreak section, which limits the team's contribution to the new PW Safety Bench. The limitation of not having multimodal data is critical and unaddressed.

Reproducibility criteria

The team correctly implemented seed setting (SEED = 123) in `execute_prompts.py` to ensure that model outputs are consistent across runs. This is good practice for scientific evaluation and determinism in future experiments.

The scripts reference relative paths like `prompts/jailbreakBench_jbb` and `dummy_data`. For full reproducibility, the repository must either include these datasets or provide a script to download/generate them automatically.

The script `evaluate.py` imports local modules that were provided in `pyproject.toml` and `uv.lock` files which provides correct dependencies needed in the project.

The inclusion of `.env.template` is a best practice for handling API keys (HF_TOKEN, GEMINI_API_KEY) without committing secrets to the repository which is included in the project and stands for good environment management.

Clean code comments

The use of the Factory Pattern in `evaluate.py` (`DataFactory`, `LLMFactory`, `JudgeFactory`) indicates a mature architectural approach which stands for modular design of the project.

The code in `execute_prompts.py` follows Pythonic standards with clear function names, helpful docstrings

Magic strings are largely avoided by defining configuration constants at the top of files (e.g., `MODELS`, `JUDGE_CONFIG`, `FIG_SIZE`).

The `try...except` blocks in `evaluate.py` prevent the entire pipeline from crashing if a single model call fails. However, printing `f"ERROR: {e}"` to the output file is a basic way to handle it; logging to a dedicated error log might be cleaner for larger runs.

The notebooks (`prompt_injection_poc_eda.ipynb`) are organized into markdown and code cells. However, ensuring all cells are cleared or pre-run in a linear order before submission would improve the "cleanliness" for the end-user.

Review authors: Julia Dudzinska, Weronika Gozdera, Marek Mytkowski, Wojciech Michaluk